

Guía de Implementación

Proyecto: Automatización de Pruebas con Cucumber, Java y Maven

Grupo 02: Doménica Cardenas, Alison Lita, Danna Morales, Salma Morales, Evelin Rocha y Genesis Vasconez

Asignatura: Verificación y Validación de Software

Fecha:

1. Introducción

Las pruebas automatizadas son una parte fundamental del desarrollo de software, ya que permiten verificar de forma rápida y repetitiva que una aplicación funcione correctamente. Una de las herramientas más utilizadas para este propósito es **Cucumber**, la cual implementa la metodología **BDD (Behavior Driven Development)**, permitiendo describir el comportamiento esperado de una aplicación mediante lenguaje natural.

En esta guía se presenta el proceso de implementación de un proyecto básico utilizando **Java, Maven, JUnit y Cucumber**, cuyo objetivo consiste en automatizar una prueba para verificar la suma de dos números.

2. Objetivo

Implementar un proyecto de automatización de pruebas utilizando Cucumber, Java y Maven, creando un escenario de prueba, sus Step Definitions y ejecutándolo correctamente mediante Maven.

3. Requisitos Previos

Antes de comenzar con la implementación se recomienda contar con lo siguiente:

- Java JDK 11 o superior.
 - Apache Maven instalado y configurado.
 - Visual Studio Code o IntelliJ IDEA.
 - Extensiones para desarrollo en Java (si se utiliza VS Code).
 - Conexión a Internet para descargar las dependencias.
-

4. Herramientas Utilizadas

Durante la implementación del proyecto se utilizaron las siguientes herramientas:

- Java
- Apache Maven

- Cucumber
- JUnit
- Visual Studio Code

5. Implementación del Proyecto

Paso 1. Crear el proyecto

Se creó un proyecto Maven llamado **CalculadoraCucumber**.

La estructura inicial del proyecto quedó organizada de la siguiente forma:

```
CalculadoraCucumber
├── pom.xml
└── src
    ├── main
    │   └── java
    ├── test
    │   ├── java
    │   └── resources
```

Explicación

Maven organiza automáticamente los proyectos Java mediante una estructura estándar. Esto facilita la administración del código fuente, las pruebas y las dependencias del proyecto.

Paso 2. Configurar el archivo pom.xml

Posteriormente se configuró el archivo **pom.xml**, agregando las dependencias necesarias para trabajar con Cucumber.

Las dependencias utilizadas fueron:

- cucumber-java
- cucumber-junit
- junit

Explicación

Cada dependencia cumple una función específica dentro del proyecto:

- **cucumber-java** permite utilizar las anotaciones **@Dado**, **@Cuando** y **@Entonces**.
- **cucumber-junit** integra Cucumber con JUnit para ejecutar los escenarios de prueba.
- **JUnit** permite validar automáticamente que el resultado obtenido sea igual al esperado mediante **assertEquals**.

Gracias a Maven, estas dependencias se descargan automáticamente cuando se ejecuta el proyecto.

Paso 3. Crear la estructura de carpetas

Dentro del proyecto se creó la siguiente estructura:

```
src
├── test
│   ├── java
│   │   ├── runner
│   │   └── steps
│   └── resources
│       └── features
```

Explicación

Cada carpeta tiene un propósito específico:

- **features** almacena los escenarios escritos en Gherkin.
- **steps** contiene el código Java que implementa cada uno de los pasos del escenario.
- **runner** contiene la clase encargada de ejecutar Cucumber.

Esta organización facilita el mantenimiento del proyecto y sigue la estructura recomendada para proyectos Maven.

Paso 4. Crear el archivo Feature

Dentro de la carpeta **features** se creó el archivo:

```
calculadora.feature
```

En este archivo se definió el escenario que verifica la suma de dos números utilizando el lenguaje Gherkin.

El escenario está compuesto por tres etapas:

- Dado
- Cuando
- Entonces

Explicación

El archivo Feature describe el comportamiento esperado del sistema utilizando lenguaje natural.

- **Dado** representa el estado inicial del sistema.
- **Cuando** representa la acción que realizará el usuario.
- **Entonces** verifica que el resultado obtenido sea el esperado.

La principal ventaja de este archivo es que puede ser comprendido tanto por desarrolladores como por testers e incluso usuarios sin conocimientos técnicos.

Paso 5. Crear los Step Definitions

Dentro de la carpeta **steps** se creó la clase:

```
CalculadoraSteps.java
```

Esta clase contiene la implementación en Java correspondiente a cada uno de los pasos definidos en el archivo Feature.

Se declararon tres variables:

- numeroA
- numeroB
- resultado

Posteriormente se implementaron tres métodos utilizando las anotaciones:

- @Dado
- @Cuando
- @Entonces

Explicación

El método anotado con **@Dado** recibe y almacena los dos números del escenario.

El método **@Cuando** realiza la operación matemática de suma entre ambos valores.

Finalmente, el método **@Entonces** compara el resultado obtenido con el resultado esperado utilizando el método `assertEquals`.

Si ambos valores coinciden, la prueba es exitosa; en caso contrario, se considera una prueba fallida.

Paso 6. Crear el Test Runner

Dentro de la carpeta **runner** se creó la clase:

```
TestRunner.java
```

Esta clase permite ejecutar los escenarios de Cucumber.

Para ello se utilizaron las anotaciones:

- @RunWith(Cucumber.class)
- @CucumberOptions

Explicación

La anotación **@RunWith** indica que JUnit ejecutará las pruebas utilizando Cucumber.

La anotación **@CucumberOptions** permite configurar:

- La ubicación de los archivos **.feature**.
- La ubicación de los Step Definitions.
- El plugin encargado de generar un reporte de ejecución.

Gracias a esta configuración, Cucumber puede localizar correctamente todos los archivos necesarios para ejecutar la prueba.

Paso 7. Ejecutar el proyecto

Una vez implementados todos los archivos, se abrió una terminal en la carpeta raíz del proyecto y se ejecutó el siguiente comando:

```
mvn test
```

Explicación

Este comando realiza automáticamente las siguientes tareas:

- Descarga las dependencias si aún no existen.
- Compila el proyecto.
- Ejecuta todos los escenarios de Cucumber.
- Genera un reporte con los resultados obtenidos.

6. Resultado Esperado

Si la implementación fue realizada correctamente, la consola mostrará un resultado similar al siguiente:

```
1 Scenario (1 passed)

3 Steps (3 passed)

BUILD SUCCESS
```

Este resultado indica que:

- El escenario fue ejecutado correctamente.
- Todos los pasos finalizaron sin errores.
- La compilación del proyecto fue exitosa.
- La automatización se ejecutó correctamente.

7. Posibles Errores

Durante la implementación pueden presentarse algunos errores comunes.

No step definitions

Se produce cuando el texto definido en las anotaciones no coincide exactamente con el texto escrito en el archivo `.feature`.

Cannot find symbol

Generalmente ocurre porque falta importar alguna dependencia o alguna clase de Cucumber o JUnit.

Maven no reconocido

Indica que Maven no está instalado correctamente o no fue agregado a las variables de entorno del sistema.

Ruta incorrecta del archivo Feature

Ocurre cuando la ruta especificada en `@CucumberOptions` no coincide con la ubicación real del archivo `.feature`.